Part 1: Class `DoubleNode`

A node is a **class**, not a `struct`. Every field is declared **separately**, with its own access modifier. The header `include/double_node.hpp` is given:

```
class DoubleNode
{
   private: NodeValue   value;
   private: DoubleNode *prevPtr;
   private: DoubleNode *nextPtr;

   public:  DoubleNode(NodeValue newValue);

   public:  NodeValue   getValue() const;
   public:  DoubleNode *getPrev() const;
   public:  DoubleNode *getNext() const;

   public:  void setPrev(DoubleNode *newPrevPtr);
   public:  void setNext(DoubleNode *newNextPtr);

   // given: copying a node is forbidden (explained in a later lecture)
   public:  DoubleNode(const DoubleNode &) = delete;
   public:  DoubleNode &operator=(const DoubleNode &) = delete;
};
```

Reading the constructor declaration

```
explicit DoubleNode(NodeValue newValue);
//                  ^^^^^^^^^ ^^^^^^^^
//                  type      parameter name
```

- `NodeValue` is the type `std::variant<int, double, std::string>`. The node can hold an `int`, a `double` or a `std::string`.
- `newValue` is the value the caller wants to store. It is passed **by value**, so the constructor receives its own copy.
- The parameter is called `newValue`, not `value`, because the field is already called `value`. Two different things must not share one name.
- In the `.cpp` the definition starts with the class name and `::`:

```
DoubleNode::DoubleNode(NodeValue newValue)
    : /* TODO: initialize all three fields */
{
}
```

Calling it: `new DoubleNode(NodeValue{5})`, `new DoubleNode(NodeValue{2.5})`, `new DoubleNode(NodeValue{std::string("abc")})`.

Your task: implement the constructor, three getters and two setters in `src/double_node.cpp`.

- The constructor must initialize **all three fields** (use an initializer list). `value` gets `newValue`. A new node has no neighbours, so `prevPtr` and `nextPtr` are `nullptr`.
- Getters are `const`: they do not modify the node.
- A node does not know anything about the list. It does not print, search or delete.

Part 2: Class `DoublyLinkedList`

The list **owns** its nodes (composition). The header `include/doubly_linked_list.hpp` is given. It contains only a **forward declaration** `class DoubleNode;`. The real `#include "double_node.hpp"` belongs in your `.cpp`.

```
#include <optional>
#include "node_value.hpp"

class DoubleNode;                              // forward declaration

class DoublyLinkedList
{
   private: DoubleNode *headPtr;
   private: DoubleNode *tailPtr;
   private: int         count;

   // internal building blocks: the caller never sees a node
   private: DoubleNode *detachFirst();
   private: DoubleNode *detachLast();
   private: DoubleNode *detachValue(NodeValue value);

   public:  DoublyLinkedList();
   public:  ~DoublyLinkedList();

   public:  void pushFrontValue(NodeValue value);
   public:  void pushBackValue(NodeValue value);

   public:  std::optional<NodeValue> popFrontValue();
   public:  std::optional<NodeValue> popBackValue();
   public:  bool deleteValue(NodeValue value);

   public:  bool containsValue(NodeValue value) const;
   public:  int  printForward() const;
   public:  int  printBackward() const;
   public:  int  length() const;

   // given: copying a list is forbidden
```

```
    public:  DoublyLinkedList(const DoublyLinkedList &) = delete;
    public:  DoublyLinkedList &operator=(const DoublyLinkedList &) = delete;
};
```

Composition rule

> The outside world works only with **values**. No public method may take, return or expose a `DoubleNode*`. Every `new` and every `delete` of a node happens **inside** the class.

Private methods

Method	Behavior
`detachFirst`	Unlink the first node and **return it**. Do not delete it. Fix `headPtr`, `tailPtr`, `count`. Clear the node's own `prevPtr` and `nextPtr`. Return `nullptr` if the list is empty.
`detachLast`	Same for the last node. Use the tail's `prev` link, so no walking is needed.
`detachValue`	Find the **first** node equal to `value` and unlink it. It can be the head, the tail, the only node or a middle node. Return `nullptr` if not found. Reuse `detachFirst` and `detachLast` where you can.

The caller of a `detach*` method owns the returned node and must `delete` it.

Public methods

Method	Behavior
`DoublyLinkedList()`	Empty list: `headPtr` and `tailPtr` are `nullptr`, `count` is `0`.
`~DoublyLinkedList()`	Delete **all** nodes (RAII). Save the next pointer **before** `delete`.
`pushFrontValue`, `pushBackValue`	Create a node with `new` and link it. Handle the **empty list** (the new node is both head and tail).
`popFrontValue`, `popBackValue`	Detach a node, read its value, `delete` the node, return the value. If the list is empty, return `std::nullopt`.
`deleteValue`	Remove the **first** node equal to `value` and free it. Return `true` if something was removed.

Method	Behavior
`containsValue`	Return `true` if any node holds `value`.
`printForward`	Print values from head to tail with `std::visit`. Return the number of printed values.
`printBackward`	Print values from tail to head using `prevPtr`. Return the number of printed values.
`length`	Return `count`.

Using `std::optional`

`std::optional<NodeValue>` is a box that holds **either a `NodeValue` or nothing**. It is a sibling of `std::variant`: with `variant` you check **which type** is inside, with `optional` you check **whether anything** is inside.

```
#include <optional>

std::optional<NodeValue> full = NodeValue{5};   // a value was assigned
std::optional<NodeValue> empty;                 // nothing was assigned

full.has_value();     // true
empty.has_value();    // false

full.value();         // NodeValue{5}, the value inside the box
```

- Assign a value, and `has_value()` is `true`.
- Assign nothing, and `has_value()` is `false`.
- `empty = std::nullopt;` puts the box back to the "nothing" state.

Check first, then open the box:

```
if (full.has_value())
{
    NodeValue v = full.value();
}
```

> Calling `.value()` on an empty `optional` stops the program. Always ask `has_value()` first, unless you are certain a value is there.

In this assignment: a function whose return type is `std::optional<NodeValue>` does one of two things.

- It has a value: `return value;` (the box is full).
- It has nothing: `return std::nullopt;` (the box is empty).

Watch out

- `NodeValue{1}` and `NodeValue{1.0}` are **not equal**: `variant` compares the active type first, then the value.
- The fields of `DoubleNode` are private. The list reads and changes links only through `getPrev`, `getNext`, `setPrev` and `setNext`.
- Deleting the only node must set **both** `headPtr` and `tailPtr` to `nullptr`.
- Removing the head: the new head's previous link must become `nullptr`. Removing the tail: the new tail's next link must become `nullptr`.
- Removing a middle node needs **two** link repairs. Forget one and `printBackward` or `popBackValue` breaks later, far from the real bug.
- Never read a node after `delete`. Read what you need (`getNext()`, `getValue()`) first.

Example:

```
void test_pop_order()
{
    DoublyLinkedList list;
    list.pushBackValue(NodeValue{1});
    list.pushBackValue(NodeValue{2});
    list.pushBackValue(NodeValue{3});

    TEST_ASSERT_(list.popBackValue().value() == NodeValue{3});
    TEST_ASSERT_(list.popFrontValue().value() == NodeValue{1});
    TEST_ASSERT_(list.length() == 1);
}

void test_pop_on_empty_list()
{
    DoublyLinkedList list;

    TEST_ASSERT_(!list.popFrontValue().has_value());
    TEST_ASSERT_(!list.popBackValue().has_value());
}
```

Tests for the wrapper

When Part 2 passes, copy the same 6 tests and change only the class name from `DoublyLinkedList` to `StdDoublyLinkedList`. Name them with the suffix `_std`, for example `test_pop_order_std`.

> If the same tests pass for both classes, the two classes implement the same interface. That is the point of this assignment.

Part 3: Class `StdDoublyLinkedList`

In Part 2 you wrote the machinery yourself: nodes, pointers, `new`, `delete`. The C++ standard library already contains a doubly linked list: `std::list`. In this part you **do not build a list**. You build a **wrapper**: a thin class that holds a `std::list` and makes it behave **exactly like your `DoublyLinkedList`**.

```
Outside:   StdDoublyLinkedList   looks the same as   DoublyLinkedList
Inside:    your own nodes and pointers          vs.   one std::list
```

The caller must not be able to tell the two classes apart. The same calls, the same results:

```
DoublyLinkedList        a;    a.pushBackValue(NodeValue{1});
StdDoublyLinkedList     b;    b.pushBackValue(NodeValue{1});
```

Why there are no private helper methods

`DoublyLinkedList` has private `detachFirst`, `detachLast` and `detachValue`, because **you** own the pointers and need building blocks to repair them.

`StdDoublyLinkedList` has **none** of them. `std::list` hides all pointer work inside itself. You never see a node, so there is nothing to detach. Your class has **one private field** and the **same public methods** as Part 2:

```
#include <list>
#include <optional>
#include "node_value.hpp"

class StdDoublyLinkedList
{
   private: std::list<NodeValue> data;

   public:  void pushFrontValue(NodeValue value);
   public:  void pushBackValue(NodeValue value);

   public:  std::optional<NodeValue> popFrontValue();
   public:  std::optional<NodeValue> popBackValue();
   public:  bool deleteValue(NodeValue value);

   public:  bool containsValue(NodeValue value) const;
   public:  int  printForward() const;
   public:  int  printBackward() const;
   public:  int  length() const;
};
```

Note what is **missing**: no constructor, no destructor, no `detach*`, no `= delete`. `std::list` cleans up after itself and copies itself correctly, so the compiler-generated versions are enough. You write none of them.

The real task: discover the library

The task is **not** to write clever code. It is to **read the documentation** and find which standard tool does the same job as each of your Part 2 methods. Every method body should be only a few lines, built from library calls.

Your method	The question you must answer
`pushFrontValue`, `pushBackValue`	Which `std::list` methods insert at either end?
`popFrontValue`, `popBackValue`	How do you read the end element and remove it? What if the list is empty?
`deleteValue`	How do you remove only the **first** element equal to `value`?
`containsValue`	How do you search without writing a loop?
`printForward`, `printBackward`	How do you go through the list from the tail?
`length`	Which method gives the size, and which type does it return?

Look things up on cppreference.com. Useful starting points:

- `std::list`: `push_front`, `push_back`, `front`, `back`, `pop_front`, `pop_back`, `empty`, `size`, `begin`, `end`, `rbegin`, `rend`, `erase`, `remove`
- `<algorithm>`: `std::find`

Rules

Forbidden in `src/std_doubly_linked_list.cpp`:

- `new`, `delete`, the class `DoubleNode`, any pointer variable (`Ptr`),
- your own `while` or `for` loops that **search or remove** elements.

Allowed:

- methods of `std::list`,
- functions from `<algorithm>`,
- iterators,
- `std::visit`,
- a loop **only for printing**.

The CI greps your file for `new`, `delete`, `DoubleNode` and `Ptr`.

Hints

- Some names look like they do the same thing as your methods. **Read the documentation, not just the name.** Two of the tools behave differently from what you need. Your tests will tell you which.

- Some `std::list` methods are **undefined behavior** on an empty list. Your public methods must still behave exactly as in Part 2.
- `size()` returns `size_t`, but `length()` returns `int`. Use `static_cast<int>(...)`.
- For printing, use `std::visit` as in `printList` from Assignment 05. Take the value **by copy**: `[](auto x) { std::cout << x << " "; }`.

  Same interface, different inside. That is what an abstract data type is. Compare the size of Part 3 with Part 2: this is how much work the library does for you.

Part 4: Your Tests and `ANSWERS.md`

Write the bodies of **8 short tests** in `src/tests.cpp` using `TEST_ASSERT_`. They are not meant to be exhaustive. Their purpose is to convince **you** that you handled the edge cases. The instructor tests in CI check correctness in depth.

Tests for `DoublyLinkedList` (6)

Test	What it should check
`test_new_list_is_empty`	`length()` is `0` on a new list
`test_push_increases_length`	after `pushFrontValue` and `pushBackValue`, `length()` is `2`
`test_pop_on_empty_list`	`popFrontValue()` and `popBackValue()` have no value
`test_pop_order`	push `1, 2, 3` with `pushBackValue`: `popBackValue()` gives `3`, `popFrontValue()` gives `1`
`test_delete_value_not_found`	`deleteValue` returns `false` and `length()` does not change
`test_delete_middle_keeps_links`	push `1, 2, 3`, delete `2`: then `popBackValue()` gives `3` and the next `popBackValue()` gives `1`

The last test is the most important one: `popBackValue` follows `prevPtr`, so it fails if you forgot to repair the backward link.

```
void test_pop_order()
{
    DoublyLinkedList list;
    list.pushBackValue(NodeValue{1});
    list.pushBackValue(NodeValue{2});
    list.pushBackValue(NodeValue{3});

    TEST_ASSERT_(list.popBackValue().value() == NodeValue{3});
    TEST_ASSERT_(list.popFrontValue().value() == NodeValue{1});
    TEST_ASSERT_(list.length() == 1);
}
```

Tests for `StdDoublyLinkedList` (2)

These two tests are about the **traps** you found in the library. Name them with the suffix `_std`.

Test	What it should check
`test_std_delete_only_first_1`	With `1, 2`, call `deleteValue(NodeValue{1})`: `length()` is 2, and `popFrontValue()` gives 2
`test_std_pop_on_empty_list`	`popFrontValue()` and `popBackValue()` on an empty list have no value and do not crash

```
void test_std_pop_on_empty_list()
{
    StdDoublyLinkedList list;

    TEST_ASSERT_(!list.popFrontValue().has_value());
    TEST_ASSERT_(!list.popBackValue().has_value());
}
```

`ANSWERS.md`

Answer each question in 2–3 sentences:

1. Count the lines in Part 2 that have **no counterpart** in Part 3: `new`, `delete`, pointer repairs. What did the standard library do for you?
2. Why does `StdDoublyLinkedList` need no destructor and no `detach*`, while `DoublyLinkedList` needs them?
3. Which two tools did **not** behave like your methods at first? What did you use instead?
4. What does `NodeValue{1} == NodeValue{1.0}` return, and why?

Rules

- Do **not** modify the headers in `include/` or any function **signature**.
- All code must compile under **C++20** (`-std=c++20`).
- No public method of `DoublyLinkedList` may take, return or expose a `DoubleNode*`.
- Always guard against `nullptr` before dereferencing a pointer.
- Every node created with `new` must be deleted exactly once, inside the class.
- `src/std_doubly_linked_list.cpp` must not contain `new`, `delete`, `DoubleNode` or `Ptr`, and no hand-written loop that searches or removes. CI checks it.

- Call `.value()` on a `std::optional` only after checking `has_value()` (or when the test guarantees a value). `.value()` on an empty `optional` stops the program.
- Do **not** use references (`&`) or templates in your own code. We have not covered them yet. (The `= delete` lines in the given headers are the only exception.)
- CI runs your `tests.cpp` together with the instructor tests. Both must pass.

Suggested schedule

Week	Work
1	Part 1 and Part 2, plus your 6 tests for `DoublyLinkedList`
2	Part 3, your 2 tests for the wrapper, `ANSWERS.md`

Part 3 is much shorter than Part 2, because you already know every operation.

Author block: required

```
std::string AUTHOR_NAME       = "Your Name";
std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this "
"assignment independently, except where explicitly noted and referenced. "
"Any collaboration or use of external resources has been properly cited. "
"I am fully aware of the consequences of academic dishonesty and agree "
"to abide by the university's academic integrity policy.";
```